Latent Teleportation

Forecasting Diffusion Trajectories for Faster Image Generation
Methods, Ablations, and a Serving Bottleneck Study

Lee Penkman
CuteDSL / OmniServe
leepenkman@gmail.com

August 4, 2026

Abstract

Speculative decoding accelerated autoregressive language models by exploiting a structural accident: decoding one token reads the entire weight matrix, so verifying several drafted tokens in one wider batch is nearly free. We ask whether the analogous accident exists in diffusion sampling, where a denoising step is likewise a single pass over a large transformer, and where the latent trajectory is smooth enough that the next point along it looks eminently guessable.

We evaluate two families of forecaster: a learned *latent walker* with a *gap interpolator* that drafts k cheap steps and teleports the draft onto the big model's likely trajectory, and a training-free first-order Taylor extrapolation of the trajectory's own momentum, in the spirit of TaylorSeer [Liu et al., 2025]. We additionally implement a k-nearest-neighbour trajectory cache and confidence-gating machinery, but do not include them in the end-to-end claims: those components remain system designs until they receive the same controlled image evaluation.

The main result has two parts. First, the signal is real: diffusion trajectories are highly forecastable, and a schedule-calibrated momentum rule reduces held-out trajectory error by 20–37% over first-order Taylor across horizons $k = 1 \dots 8$. An independent anchor-placement search further reduces a four-interval trajectory proxy to relL2 $= 0.215$, a 35.3% improvement over uniform Taylor. Across the 36 evaluated learned and Taylor image candidates, an author screen finds that every output retains the requested subject or scene; the visible trade is increasing layout and texture drift as k grows. Second, the present serving point does not yet turn fewer denoiser calls into a large end-to-end gain. At 512^2 and 16 steps on an RTX 5090, speculative walking reaches 1.04×, while a destructive skip control that removes 3.2× of the denoiser reaches only 1.10×. This locates the immediate bottleneck in timed text encoding and VAE decoding, and gives the next experiments a concrete target rather than closing the direction.

We further report that our first analysis of these same runs claimed 1.5×, and that this figure was an artifact of an un-warmed first iteration whose baseline took 140.6 s against a 28.3 s median. We give the corrected protocol, a reproducible analysis harness, five-fold forecaster and aligned-schedule ablations, draft-length and routing ablations, three grids containing the actual generated images across six prompt/style slices, an account of how the method extends to swarms of LoRA adapters with disjoint embedding spaces, and deployment recommendations grounded in the measured speed–quality frontier.

1 Introduction

A diffusion sampler spends nearly all of its arithmetic evaluating one large denoiser many times over. The obvious economy is to evaluate it fewer times, and the literature offers two broad routes: change the model so that fewer steps suffice — distillation [Salimans and Ho, 2022], consistency models [Song et al., 2023, Luo et al., 2023], adversarial distillation [Sauer et al.,

2023] — or leave the model alone and reuse computation across steps [Ma et al., 2024, Selvaraju et al., 2024, Liu et al., 2025]. The second route is attractive operationally because it does not require retraining a production checkpoint.

Speculative decoding [Leviathan et al., 2023, Chen et al., 2023] suggests a third framing. In language models, the win comes not from a better model but from a hardware fact: autoregressive decode is memory-bandwidth-bound, so a batch of $k+1$ candidate tokens costs about what one costs, and a cheap drafter can propose tokens that the real model merely *verifies*. The appeal is that verification is exact — the accepted output is distributed exactly as the target model's — so speculation is a pure latency optimisation rather than a quality trade.

Diffusion sampling looks superficially similar. It is iterative, each iteration is one pass over a transformer, and the sequence of latents $x_0, x_1, \ldots, x_N$ is a smooth curve rather than a sequence of discrete symbols — if anything, easier to extrapolate than text. This paper asks which parts of the analogy carry, which do not, and what must change for the forecastability signal to become useful serving latency.

What latent teleportation is — and is not

The name describes a family of approximate trajectory shortcuts: estimate a local tangent or transport map, then warp a latent along the learned manifold to a later refinement state. It is not one fixed algorithm and not an exact analogue of language-model speculative decoding.

What it is	**What it is not**
A forecast, manifold warp, or warm start in a continuous diffusion latent trajectory.	An exact verifier or a distribution-preserving decoding method.
A family that can use momentum, aligned schedules, learned predictors, confidence gates, or retrieved trajectories.	A claim that every listed mechanism has already passed image and latency evaluation.
A controllable trade between denoiser calls and same-seed image retention.	A guarantee of end-to-end speedup when text, VAE, transfer, or scheduling costs dominate.
A model–serving co-design problem whose useful operating point can change with resolution, schedule, and batching.	A single checkpoint-specific result that should be extrapolated to every sampler or deployment.

Contributions.

1. **A predictability study** of the Z-Image [Tongyi Lab, Alibaba Group, 2025] latent trajectory: a (t, k) grid of forecast error for identity, first-order Taylor and scalar-affine predictors, establishing that the trajectory is genuinely forecastable and quantifying where (Section 5.1).

2. **A five-fold forecaster ablation** showing that one fitted schedule scalar improves over raw Taylor by 20–37% through $k = 8$, while an extra historical delta adds little and an unfitted second-order rule is worse (Section 5.3).

3. **A call-budget schedule ablation** connecting trajectory forecasting to Align Your Steps: four cross-validated forecast intervals improve local forecast error by 35.3%, while four staggered near-optimal schedules remain within 2.4% of the best (Section 5.4).

4. **Two evaluated acceleration mechanisms** — speculative latent walking with a learned gap interpolator and fixed-horizon Taylor forecasting — plus clearly separated designs for confidence gating and k-NN cache teleportation (Section 3).

5. **An operating-point bottleneck diagnosis**: call reductions of 3.2× yield 1.04× wall-clock, and a skip-everything ablation caps at 1.10×, localising the bottleneck outside the denoiser (Section 5.5).

6. **A methodological correction**. Our own earlier reading of these runs reported 1.5×; we show it was warm-up contamination, and ship an analysis harness that cannot make that mistake again (Section 5.7).

7. **Image-level ablations** over learned speculation, Taylor forecasting, a destructive skip control, three draft lengths, and six prompt/style slices. We publish all panels as contact sheets with PSNR, SSIM, and source-file hashes (Section 5.6).

8. **An extension sketch to LoRA armies**, where adapters induce disjoint latent statistics and a shared trajectory cache needs an embedding normalisation network (Section 3.5).

2 Background and Related Work

Diffusion and flow samplers. We work with a rectified-flow / flow-matching formulation [Liu et al., 2023, Lipman et al., 2023] as used by Z-Image, sampled on a fixed schedule of N timesteps in a latent space produced by a VAE [Rombach et al., 2022], with a DiT-style denoiser [Peebles and Xie, 2023]. Writing Φ for one scheduler step,

$$x_{i+1} = \Phi(x_i, t_i, c), \tag{1}$$

with c the text conditioning. The cost of sampling is N evaluations of the denoiser inside Φ, plus one text encode and one VAE decode.

Step reduction by retraining. Progressive distillation [Salimans and Ho, 2022], consistency models [Song et al., 2023], latent consistency models [Luo et al., 2023] and adversarial distillation [Sauer et al., 2023] all produce a new checkpoint that needs fewer steps. LCMs explicitly target high-resolution generation in two to four steps; this is powerful but consumes the very redundancy that a post-training skip method would otherwise exploit. The combination is therefore role-dependent: a consistency model can serve as a cheap endpoint proposer or training target, while teleportation can target a longer high-quality teacher schedule. Z-Image Turbo is already distilled, which matters for interpreting our results (Section 9).

Schedule optimisation. Align Your Steps (AYS) [Sabour et al., 2024] optimises the sampling timesteps for a chosen model, dataset, and stochastic solver by minimising a bound on the mismatch between a locally linearised and true reverse process. It asks *where* expensive evaluations should occur. Latent teleportation asks *what state should fill the interval between them*. These decisions are complementary, and Section 5.4 measures their joint offline proxy rather than assuming uniform skips.

Flow matching, straightening, and reflow. Flow matching can train vector fields along diffusion or optimal-transport probability paths; the latter are straighter and admit efficient ODE sampling [Lipman et al., 2023]. Rectified flow and reflow make this connection operational by straightening noise-to-data trajectories, which in turn eases few-step distillation [Liu et al., 2023, 2024]. This is the geometry latent teleportation tries to exploit after training: straighter paths make a local tangent valid for longer, while schedule calibration absorbs the remaining non-uniform speed. Reflow changes the model; teleportation can be fitted to a frozen model. Combining them should improve predictability, but must be re-measured because a successfully straightened few-step model also leaves fewer calls to remove.

Training-free caching. DeepCache [Ma et al., 2024] reuses U-Net skip features across adjacent steps. FORA [Selvaraju et al., 2024] caches attention and MLP outputs in diffusion transformers. TaylorSeer [Liu et al., 2025] — the line our codebase calls CG-Taylor — reframes caching as *forecasting*: rather than reusing a stale feature, extrapolate it with a Taylor series in the timestep index. Crucially, these methods are fitted to a specific model: the cache schedule and the expansion order are tuned per checkpoint, and transfer is not claimed. Our approach inherits that property and we make it explicit, since it is a real deployment cost rather than an incidental detail.

Speculative decoding. Draft-and-verify [Leviathan et al., 2023, Chen et al., 2023], with later variants that avoid a separate draft model by adding heads [Cai et al., 2024], reusing features [Li et al., 2024], or drafting from the prompt itself [Saxena, 2023]. The property that makes these work is verification: the target model's distribution is preserved exactly. It is worth stating plainly that *our diffusion analogue does not have this property.* There is no cheap exact verifier for "is this latent the one the big model would have produced", so every method in this paper is an approximation with a quality cost, and must be evaluated as such.

3 Method

3.1 Speculative latent walking

The drafting scheme mirrors speculative decoding structurally. The big model takes one real step; a small network drafts the next k; a second network corrects the endpoint of the draft onto where the big model's trajectory would plausibly have gone; the loop resumes with a real step.

The **latent walker** W predicts the next latent from the current one and the trajectory's momentum $\delta_i = x_i - x_{i-1}$:

$$\hat{x}_{i+1} = x_i + \delta_i + W(x_i, \delta_i, t_i), \tag{2}$$

so that the network learns only the *correction* to constant-velocity motion. The output convolution is zero-initialised, making the network exactly a momentum extrapolator at initialisation and guaranteeing it starts no worse than the training-free baseline. Rolling W out k times, feeding it its own predicted movement, yields a draft endpoint $\hat{x}_{i+k}$.

The **gap interpolator** G then teleports that endpoint:

$$\tilde{x}_{i+k} = \hat{x}_{i+k} + G(x_i, \delta_i, \hat{x}_{i+k}, t_i, k), \tag{3}$$

again residual and zero-initialised, so identity means "trust the walker". Both networks are small FiLM-conditioned residual CNNs operating directly on the 16-channel latent, conditioned on sinusoidal embeddings of the fractional timestep and the schedule length.

3.2 First-order Taylor forecasting and confidence gating

The training-free baseline extrapolates the trajectory's own momentum:

$$\tilde{x}_{i+k} = x_i + k\,(x_i - x_{i-1}), \tag{4}$$

which is exactly a first-order Taylor step in the step index, in the spirit of Liu et al. [2025]. This is the floor every learned method must beat, and in our results it is a stubbornly high floor.

The **confidence gate** would add the CG in CG-Taylor. Rather than a fixed skip schedule, it tracks the recent history of prediction error, estimates the derivative of that error, and maps the resulting confidence onto a number of refinement steps to spend — high confidence, fewer real steps. The design intent is the diffusion analogue of forecasting one block into the network and committing to the whole step if that forecast lands: a cheap partial computation acts as a

proxy for whether the expensive full computation can be skipped. The end-to-end Taylor arm reported below uses fixed k and does not activate this gate; we therefore treat confidence gating as unvalidated system design.

3.3 Aligned anchors and selective batch lanes

A constant k assumes that equal scheduler-index gaps are equally easy to forecast. They are not. We separate the decision into an anchor schedule $A = (a_0, \ldots, a_m)$ and a transport rule T:

$$\tilde{x}_{a_{j+1}} = T(x_{a_j}, x_{a_j-1}, c, a_j, a_{j+1} - a_j). \tag{5}$$

AYS motivates choosing the anchors non-uniformly; our calibrated transport uses an empirical per-step, per-horizon speed correction. A retrieved neighbour can further supply a local tangent or curvature estimate, so similar historical trajectories adjust both the direction of the warp and the next point at which the big model should refine it. This is better understood as moving within a local chart of the latent manifold than as copying a cached image.

For a batch of B outputs, the schedule can be lane-specific: $A^{(1)}, \ldots, A^{(B)}$. At a global scheduler event, lanes requiring a real update are compacted into an active microbatch; the remaining lanes advance with T. Different noise seeds already provide ordinary batch diversity. Lane-specific neighbours, horizons, and bounded residual perturbations add *structured* variation without requiring every lane to take every denoiser call. This saves work only if the active-lane compaction costs less than the removed transformer rows; issuing a full batch and masking its outputs saves nothing. The released code includes the deterministic event planner that groups lane endpoints into compact denoiser microbatches; the GPU gather/transformer/ scatter path remains to be benchmarked. Section 7 gives the production protocol, while Section 5.4 measures only the offline schedule proxy.

3.4 Latent teleportation from a trajectory cache

The third mechanism drops speculation entirely and asks a different question: if this prompt resembles one we have generated before, can we start from where that generation *ended up* rather than from noise?

The cache stores, per generation, the full sequence of intermediate latents together with a CLIP [Radford et al., 2021] embedding of the prompt and a tokenisation of it into *visual units* — noun-phrase-like fragments that recur across prompts — plus bigrams of adjacent units. At query time the system retrieves the k nearest cached trajectories by prompt embedding, combines their latents at a chosen injection timestep (SLERP, a learned combiner, or a tree combiner), injects the result, and runs only the remaining scheduler steps. A k-NN *trajectory prior* additionally warm-starts the walker rollout with neighbour deltas.

The intuition is that similar refinements trace similar paths through the latent manifold, so a neighbourhood of finished trajectories defines a local chart that a new generation can be teleported into. Storage is SafeTensors blobs indexed by SQLite; retrieval is exact k-NN over a few thousand vectors, for which a brute-force scan suffices and Johnson et al. [2019] would be the drop-in replacement at scale.

3.5 Extension to LoRA armies

Production serving does not run one checkpoint. It runs a base model with a library of LoRA adapters [Hu et al., 2022] swapped per request — in our deployment, an army of style and subject adapters, hot-swapped between generations. This breaks the trajectory cache in a specific way: an adapter shifts the latent statistics, so a trajectory collected under adapter A is not a valid warm start under adapter B, even for an identical prompt. Naively sharing the cache produces

exactly the failure mode the confidence gate is supposed to catch, but at a magnitude that no gate rescues.

The extension we propose — and have not yet validated, so we state it as design rather than result — is an *embedding normalisation network*: a small conditional map $\eta_{A\to B}$ trained to transport a latent from adapter A's trajectory space into adapter B's, conditioned on both adapter identities and the timestep. Cheap variants worth trying first, in increasing order of capacity, are (i) per-adapter per-timestep channel-wise mean/variance renormalisation, which costs two vectors per adapter per step; (ii) a low-rank affine map fitted per adapter pair; (iii) a shared conditional network taking learned adapter embeddings, which scales to $O(n)$ parameters rather than $O(n^2)$ for n adapters. Adapter identity is exactly the kind of low-cardinality conditioning that FiLM handles well, and the zero-initialised residual trick used above applies again: initialise to identity so that a shared cache degrades to a per-adapter cache rather than to garbage.

The adapter library and its serving stack are not open-sourced; the mechanism described here is, in Penkman [2026a].

4 Experimental Setup

All measurements are on a single RTX 5090 (32 GB, sm_120, compute capability 12.0) using Z-Image Turbo in `bfloat16`, at 512×512 with a 16-step schedule and guidance 0, seed fixed at 7 across arms. Trajectory capture collected 200 full 16-step trajectories (402 MB) storing every intermediate latent. The image ablation uses six deliberately heterogeneous prompt slices: golden-hour landscape, dramatic portrait, snowy night, cyberpunk/neon, soft-light wildlife, and isometric illustration. These labels describe prompt content; they are not independent style datasets or LoRA adapters.

Metrics. Forecast quality is measured by

$$\text{relL2} = \frac{\|\tilde{x}_{i+k} - x_{i+k}\|_2}{\|x_{i+k} - x_i\|_2}, \tag{6}$$

error normalised by how far the trajectory actually moved. This normalisation is what makes the number interpretable: relL2 = 1.0 is the score of predicting no movement at all, so any method scoring near 1.0 is worthless regardless of its absolute error. End-to-end image agreement is PSNR against the full-schedule baseline image. For the image grids we additionally report SSIM, computed on the saved RGB PNGs. Both are reference-similarity metrics rather than measures of prompt adherence or aesthetic quality. Wall clock is measured with CUDA synchronisation around each arm.

Reproducibility. Timing and latent-forecast numbers are emitted by `scripts/paper_analysis.py` from raw result JSON. `scripts/spec_forecaster_ablation.py` fits and cross-validates horizons through $k = 8$ and emits full-schedule deployment coefficients; `scripts/spec_schedule_ablation.py` searches anchor compositions under fixed budgets. Image tables and grids are emitted by `scripts/paper_ablation_grids.py` from the 72 saved PNGs. The latter writes a manifest containing the SHA-256 digest, prompt index, method, k, PSNR, and SSIM for every source panel. No panel is synthesized or retouched; grid construction is limited to resizing, borders, and labels.

5 Results

5.1 The latent trajectory is forecastable

Table 1 summarises the (t, k) grid of forecast error. Two facts stand out. First, first-order Taylor extrapolation is strong: at $k = 1$ it reaches relL2 = 0.205 around step 6, meaning the residual error is about a fifth of the movement being predicted. Second, the scalar-affine predictor sits near 0.88 — barely better than not moving — which tells us the trajectory's predictability is genuinely directional rather than a matter of getting the step size right.

k	mean relL2 (taylor1)	best cell	mean relL2 (affine)	cells
1	0.240	0.205	0.880	13
2	0.318	0.255	0.879	13
3	0.330	0.261	0.883	12
4	0.369	0.261	0.880	11

Table 1: Trajectory predictability by draft length. relL2 is error over actual movement; 1.0 is the no-movement baseline. Lower is better. The degenerate final-step cell is excluded (see text).

Error degrades monotonically with k, as expected, and is worst at both ends of the schedule: early steps move erratically, and the last scheduler step of this 16-step configuration moves the latent by approximately zero. That final step is not merely hard to predict — it is a no-op, and dividing by its movement produced an relL2 of 5.48×10^9 at $t = 14$. We exclude that cell from all aggregates. In the stored float16 trajectories, the final latent is duplicated; for this exact pipeline and schedule, that step is a candidate for exact removal after verifying the duplication at runtime.

5.2 Learned walker versus training-free Taylor

Training the walker and interpolator for 12 epochs on the 200 captured trajectories reaches a best relL2 of 0.242 for the walker, against roughly 0.21–0.30 for Taylor over the comparable window — an improvement of order 15–20% on the metric it was trained for. Momentum conditioning is what produces this; an earlier position-only walker plateaued around 0.6, worse than Taylor.

The training is not clean, and we report it rather than presenting the best epoch as if it were the endpoint. The walker's final-epoch score is 0.257 and the interpolator's is 0.298, both worse than their best; a run that saves the last checkpoint therefore ships a model worse than the one it trained. Any redeployment of this method should select on validation relL2 rather than take the final epoch.

5.3 Schedule calibration improves the cheap forecaster

The basic Taylor rule assumes that one recorded movement should be multiplied by exactly k. Diffusion schedules are not uniform in trajectory distance, so we fit a separate scalar $\alpha_{t,k}$ for each step and horizon:

$$\hat{x}_{t+k} = x_t + \alpha_{t,k}\, k\, (x_t - x_{t-1}). \tag{7}$$

We also test a two-delta fit $x_t + a_{t,k}(x_t - x_{t-1}) + b_{t,k}(x_{t-1} - x_{t-2})$, an average-velocity rule, and a direct second-order Taylor extrapolation. Coefficients are fitted on four folds and evaluated on the held-out fifth, rotating over all five folds of the 200 captured trajectories. Cells whose target movement is below 10^{-4} are excluded by the same rule as Section 5.1.

Table 2 gives a constructive tuning result. Scaled momentum reduces error by 20–37% relative to first-order Taylor across $k = 1 \ldots 8$, with the largest relative gain at $k = 8$ (36.7%). The two-delta fit remains only marginally better: at $k = 8$ it reaches 0.327 against 0.330 for one

k	Taylor-1	avg. velocity	Taylor-2	scaled momentum	two-delta fit
1	0.221±0.005	0.273±0.004	0.226±0.009	0.176±0.007	0.172±0.007
2	0.306±0.004	0.338±0.004	0.370±0.011	0.205±0.007	0.201±0.007
3	0.317±0.004	0.352±0.004	0.395±0.013	0.228±0.008	0.226±0.008
4	0.357±0.004	0.393±0.004	0.446±0.016	0.250±0.009	0.248±0.008
5	0.402±0.004	0.435±0.004	0.505±0.018	0.271±0.009	0.269±0.009
6	0.445±0.004	0.473±0.003	0.567±0.020	0.291±0.010	0.289±0.010
7	0.484±0.004	0.510±0.003	0.628±0.021	0.310±0.010	0.309±0.010
8	0.521±0.003	0.544±0.003	0.689±0.022	0.330±0.011	0.327±0.011

Table 2: Five-fold held-out relL2 (mean ± standard deviation across folds). Lower is better. Scaled momentum fits one schedule scalar; two-delta fits two. No image pixels or timing labels enter the fit.

scalar. The second delta therefore contributes little even at long horizons, making the one-scalar rule the better first deployment candidate. Average velocity and raw second-order Taylor are both worse than first order: more history or a higher expansion order is not automatically better unless it is calibrated to the schedule. This is a held-out latent-level result, not yet an image-quality or latency claim; the next image sweep should promote scaled momentum alongside the existing Taylor reference.

5.4 Aligning anchors improves a fixed interval budget

We next hold the number of forecast intervals fixed and search their integer horizons. The proxy covers recorded steps 2 through 14, excluding the duplicated final latent. For each outer fold, coefficients and a schedule are selected on four folds and evaluated on the fifth. Each interval starts from its recorded true anchor; the experiment therefore measures local anchor placement and does not simulate error accumulated by a branched sampler.

intervals	uniform Taylor	uniform calibrated	aligned calibrated	four staggered	selected horizons
3	0.380±0.004	0.270±0.009	0.248±0.008	0.254±0.008	8–1–3
4	0.332±0.005	0.243±0.008	0.215±0.007	0.220±0.007	8–1–1–2
6	0.280±0.005	0.212±0.008	0.188±0.006	0.190±0.006	7–1–1–1–1–1

Table 3: Five-fold anchor-schedule ablation. The budget counts forecast intervals, not denoiser calls. Entries are mean local relL2 ± standard deviation. Four staggered reports the mean of the four best distinct training-fold schedules, modelling four batch lanes. Selected horizons are stable across all five folds. Lower is better.

Table 3 separates two gains. With four intervals, calibration alone improves uniform scheduling from 0.332 to 0.243. Aligning the same interval budget further improves it to 0.215—35.3% below uniform Taylor and about 12% below uniform calibrated momentum. Every fold selects $[8, 1, 1, 2]$: one long early warp followed by dense late corrections. That pattern is consistent with a rectified-flow path that is relatively straight early but becomes more sensitive as image details settle; it is an empirical result for this schedule, not a universal timetable.

The four best distinct schedules average 0.220, only 2.4% above the single best schedule. This is the first quantitative support for staggered batch lanes: four outputs can use different anchor patterns with little latent-proxy penalty. It does not yet establish either decoded-image diversity or GPU speedup; both require active-lane compaction and the image protocol in Section 7.

5.5 End-to-end: locating the remaining latency

Table 4 measures the current serving point. Across draft lengths $k \in \{1, 2, 3\}$, corresponding to 9, 6 and 5 real denoiser calls out of 16, measured speedup never exceeds 1.10× and speculative walking specifically never exceeds 1.04×.

k	big steps	call red.	wall-clock speedup			PSNR vs baseline (dB)		
			spec	taylor	skip	spec	taylor	skip
1	9	1.78×	1.02	1.08	1.03	18.9	20.8	11.0
2	6	2.67×	1.03	1.03	1.04	18.0	17.9	9.3
3	5	3.2×	1.04	1.09	1.10	17.0	16.9	8.9

Table 4: Draft-length ablation at 512^2, 16 steps, 5 prompts per cell after warm-up exclusion. "call red." is the reduction in denoiser evaluations; speedup is measured wall clock. *skip* performs no correction at all and therefore bounds what any drafting scheme can achieve in this harness.

The diagnostic is the *skip* column. That arm discards 3.2× of the denoiser evaluations and replaces them with nothing — its PSNR of 8.9 dB confirms the output is destroyed — and it still only reaches 1.10×. Its observed ceiling is therefore about 1.10× at this operating point, implying that roughly 90% of the measured time is outside the removed calls. Inspecting the harness confirms it: the timed region wraps `zimage_prepare` (text encoding) and `zimage_decode` (VAE decode) along with the sampling loop, and at 512^2 with a 16-step Turbo schedule those fixed costs dominate. This is Amdahl's law [Amdahl, 1967] arriving exactly on schedule.

A second observation from Table 4: at $k = 1$ the training-free Taylor arm is both faster (1.08× vs 1.02×) and higher quality (20.8 dB vs 18.9 dB) than the learned walker. The learned networks win on the latent metric they were trained for but lose on same-seed pixel retention. This sets a useful tuning reference: Taylor at $k = 1$ is the current image-level baseline, and a learned replacement needs an image-aligned objective and validation-based checkpoint selection to displace it.

5.6 Image ablations across methods, styles, and draft lengths

The aggregate timing result is bottleneck-limited, and the images answer the complementary question of *how* each approximation changes the sample. Figures 1, 2, and 3 expose the entire recorded image sweep rather than one favourable prompt. The raw store contains 72 PNGs: six prompts, four arms, and three values of k. Because the baseline is regenerated identically for every k, this corresponds to six unique references and 54 candidate outputs. The figures include all six prompt slices, including the first timing row excluded as a warm-up outlier; warm-up affects latency, not the already-saved image.

k	real calls	learned		Taylor		skip	
		PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
1	9	18.14	0.699	19.78	0.770	11.11	0.244
2	6	17.28	0.662	17.29	0.637	9.35	0.182
3	5	16.51	0.629	16.39	0.585	8.95	0.173

Table 5: Image hyperparameter ablation over all six prompts. Values are mean PSNR (dB) and SSIM against the same-seed 16-step reference. Unlike Table 4, no row is removed because this table contains no timing measurement. Higher is better.

Table 5 gives three robust observations. First, $k = 1$ is the quality frontier: Taylor reaches 19.78 dB/0.770 and the learned method 18.14 dB/0.699. Second, mean similarity falls as k grows. Learned SSIM moves $0.699 \to 0.662 \to 0.629$, while Taylor moves $0.770 \to 0.637 \to 0.585$. Third, the skip control is not a plausible alternative generation: even at $k = 1$ it reaches only 0.244 SSIM and visually collapses to coloured noise. It is useful only as the compute-saving upper bound in Table 4.

The style-stratified $k = 3$ numbers in Table 6 show why six rows matter. Learned speculation is ahead of Taylor on portrait, snowy-night, and isometric prompts; Taylor is ahead on landscape, cyberpunk, and wildlife by PSNR. The ranking is therefore three against three rather than a universal winner. Structural retention also varies widely: learned SSIM ranges from 0.500 on the snowy scene to 0.791 on the isometric illustration. These are descriptive slices, not confidence intervals, but they rule out a claim that one forecaster dominates across visual regimes.

style slice	learned		Taylor		skip	
	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
Golden-hour landscape	13.87	0.713	14.12	0.666	9.32	0.165
Dramatic portrait	19.11	0.513	17.72	0.453	7.37	0.096
Snowy night scene	15.26	0.500	14.68	0.454	8.82	0.177
Cyberpunk / neon	14.70	0.504	15.74	0.506	9.33	0.217
Soft-light wildlife	18.53	0.751	18.82	0.692	11.79	0.284
Isometric illustration	17.61	0.791	17.28	0.738	7.08	0.097

Table 6: Style-slice ablation at $k = 3$ over the six recorded prompts. Each row contains one image per method, so the values describe examples rather than a population estimate.

An oracle that independently chooses the higher-scoring learned or Taylor image for each prompt estimates the available headroom from method routing. It is not a deployable router because it sees the reference image and chooses PSNR and SSIM independently.

	PSNR (dB)			SSIM		
k	learned	Taylor	oracle	learned	Taylor	oracle
1	18.14	19.78	19.78	0.699	0.770	0.772
2	17.28	17.29	17.62	0.662	0.637	0.665
3	16.51	16.39	16.78	0.629	0.585	0.629

Table 7: Reference-aware oracle routing between the learned and Taylor candidates. The oracle is an unavailable upper bound, not an evaluated routing algorithm. Higher is better.

Table 7 shows modest routing headroom. At $k = 1$, Taylor already wins PSNR on every prompt; at $k = 2$ and $k = 3$, oracle selection adds only 0.33 and 0.38 dB over the better fixed method. Oracle SSIM is similarly close to the best fixed choice. Schedule calibration is therefore the more promising immediate experiment; a learned router becomes worthwhile only when the candidate methods are more complementary.

The grids also sharpen the metric caveat. A coherent candidate that moves a subject or changes texture can score poorly against the reference, while SSIM rewards structurally simple scenes differently from detailed photographs. We therefore use these metrics to quantify same-seed retention, not to assert equal aesthetic quality. A blinded human or calibrated vision-language preference study remains required for that claim.

We additionally inspected every full-resolution source panel, not only the contact sheets. In this unblinded author screen, all 36 learned and Taylor candidates retained the requested subject or scene, with no catastrophic collapse; increasing k primarily introduced texture, geometry, and layout drift. None of the 18 skip candidates passed the same usable-image screen, although several $k = 1$ panels retained fragments of scene structure. This is a failure-mode check, not a blinded aesthetic or prompt-adherence evaluation.

5.7 A correction to our own earlier numbers

Our first analysis of these runs reported a mean speedup of 1.5× at $k = 3$. That number is wrong, and the way it was wrong is instructive.

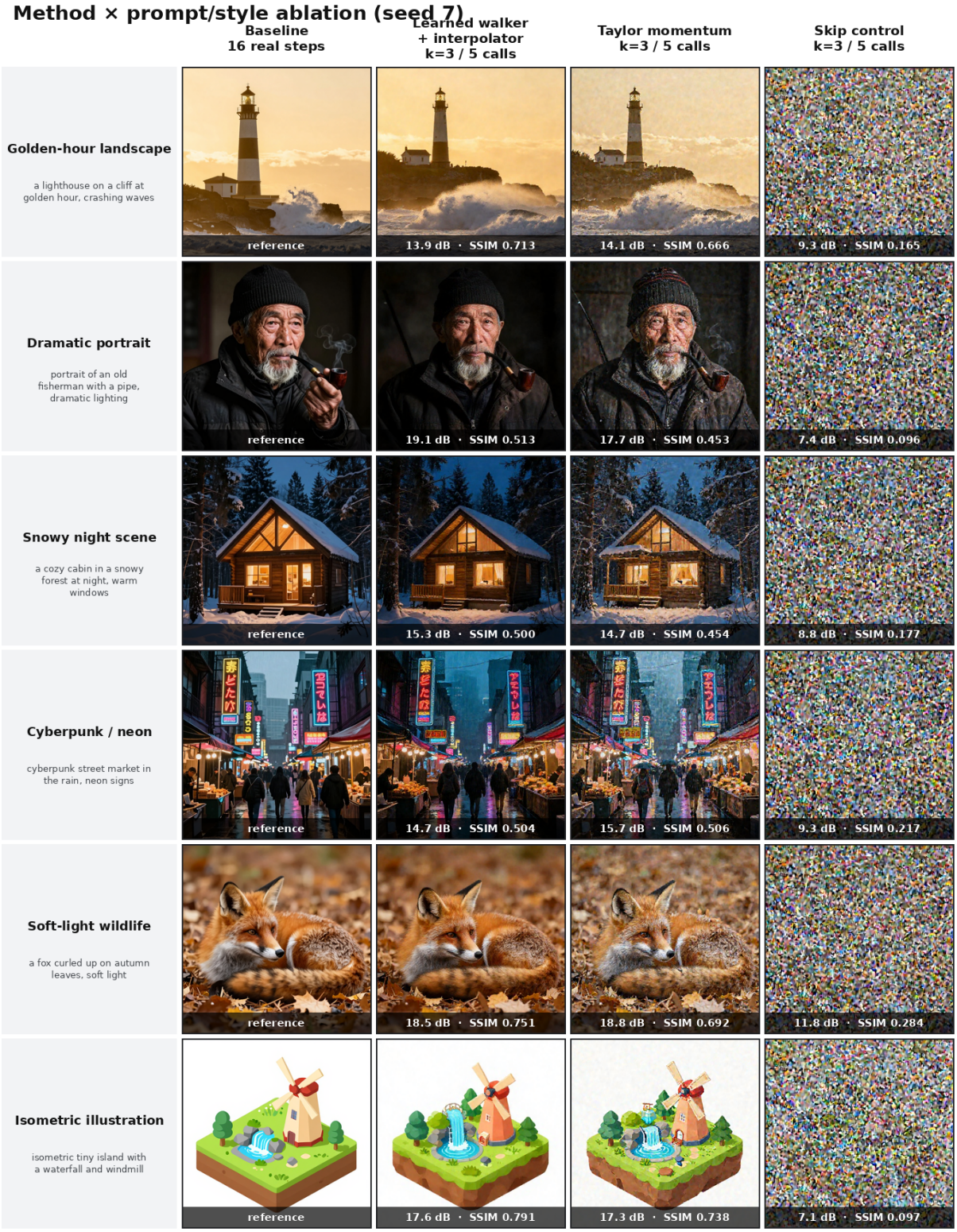

Figure 1: Method × prompt/style ablation at $k = 3$ (five real denoiser calls), seed 7. Each row is one saved prompt; each panel is an actual generated PNG with no content editing. The learned and Taylor arms usually preserve the requested subject while changing layout, texture, or fine detail. The skip control is visibly destructive in every slice.

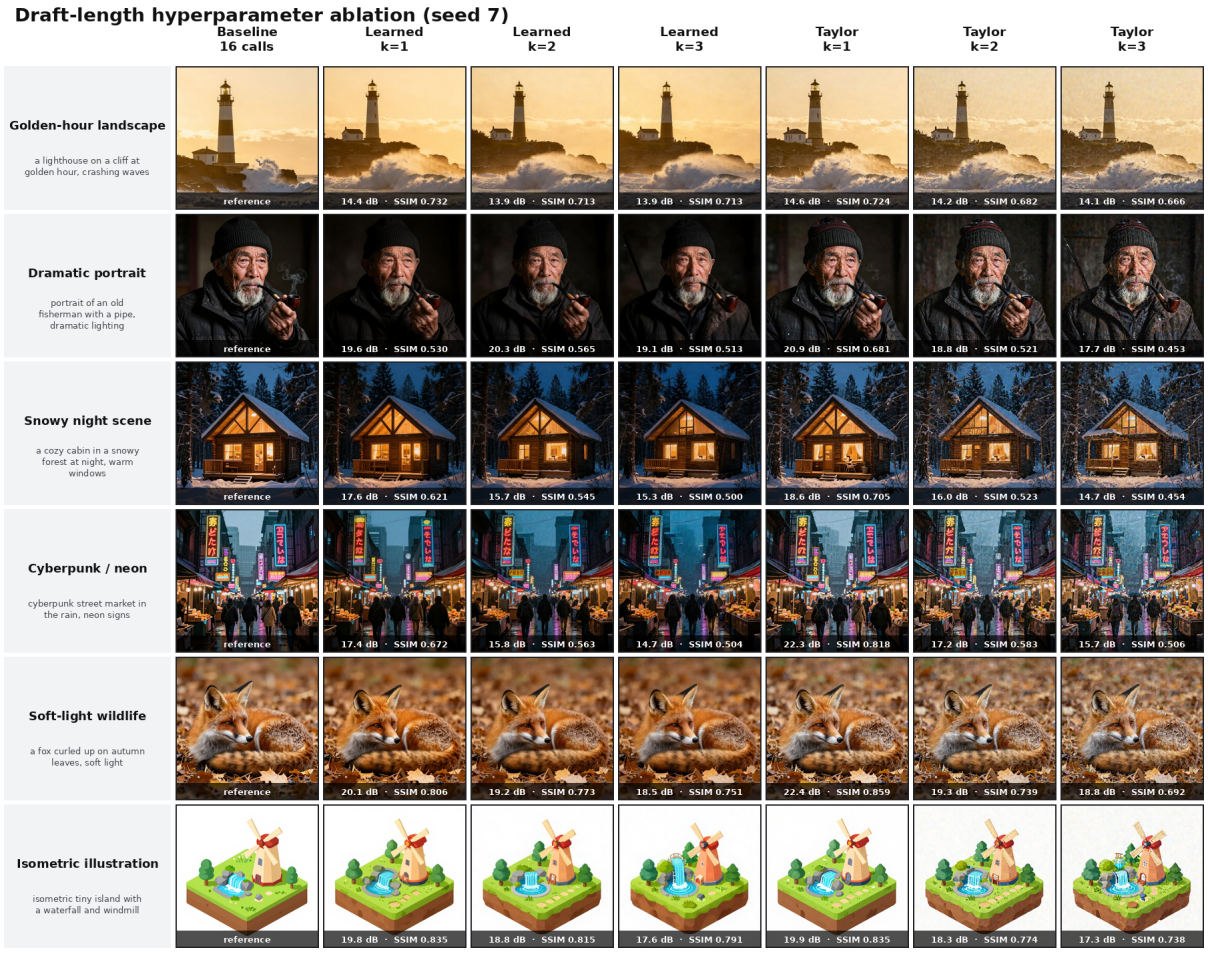

Figure 2: Draft-length hyperparameter ablation. Columns vary forecaster family and $k \in \{1, 2, 3\}$; rows vary prompt/style slice. Increasing k means fewer real denoiser calls (9, 6, and 5 respectively) and progressively greater departure from the same-seed reference. Panel labels report PSNR and SSIM.

Figure 3: Skip-control hyperparameter ablation. This arm performs the same real denoiser-call schedules as the learned and Taylor arms but supplies no latent correction. Its outputs deteriorate from weak structure at $k = 1$ to coloured noise at $k = 3$, demonstrating that call removal alone does not yield a viable sample.

The harness times the baseline arm first, for each prompt, with no warm-up iteration. The first baseline call therefore absorbs CUDA context creation, cuBLAS/cuDNN autotuning and lazy module initialisation: it took 140.6 s against a 28.3 s median for identical work. Because speedup is computed as $t_{\text{baseline}}/t_{\text{method}}$ per prompt and then averaged, that single contaminated row contributed a spurious 3.79× and carried the mean. The same pattern appears at $k = 1$ (96.8 s) and $k = 2$ (64.1 s), inflating those means too.

Excluding rows whose baseline exceeds twice the median — the rule our analysis script now applies and reports — the means fall to the values in Table 4. We record this because a 1.5× claim and a 1.04× measurement lead to different engineering decisions, and because the failure is entirely generic: any benchmark that times a cold arm first and averages ratios will do this.

6 Recommendations

The ablations define a focused next tuning loop.

1. **Tune from the Taylor $k = 1$ reference.** It is the best measured speed/retention point: 1.08× at 20.8 dB PSNR, with no learned checkpoint or extra model allocation.

2. **Promote schedule-calibrated momentum into the next image sweep.** Its five-fold trajectory error is 20–37% below raw Taylor through $k = 8$ and it captures nearly all of the benefit of the more complex two-delta fit. Test both uniform and aligned horizons with the same seeds before claiming an image-level gain.

3. **Use aligned rather than uniform anchors.** At a four-interval budget, schedule search contributes an additional 12% latent-proxy improvement beyond calibration. Refit the timetable for every model, resolution, solver, and step count rather than copying $[8, 1, 1, 2]$.

4. **Align learned training with the output metric.** Save the best validation checkpoint and add decoded perceptual or preference supervision; the present relative-L2 objective improves latent error without beating the Taylor image baseline.

5. **Treat routing as a second-stage optimisation.** The reference-aware oracle adds at most 0.38 dB in this sweep, so a router has less immediate headroom than improving the forecaster itself.

6. **Test removal of the final step for this exact schedule.** The stored float16 trajectory duplicates its final latent. Gate the optimisation on a runtime equality/tolerance check rather than generalising it to other schedulers or step counts.

7. **Report denoiser-only and end-to-end timing together.** Warm up every arm, cache prompt embeddings where the serving workload permits it, and separate sampling from VAE decode. This reveals both the method gain and the user-visible gain.

8. **Expand the operating-point grid** to higher resolution, longer or undistilled schedules, and batches greater than one. Those settings give saved denoiser calls enough share of total time to matter.

7 Production Use Cases and Composability

7.1 Thirty-step quality at approximately four-call speed

The production objective is not merely to make a native four-step sample; it is to approach the quality of a 30-step teacher while paying for approximately four large-denoiser evaluations. We treat this as the next acceptance target, not as a result already established by the 16-step

study. The experiment should capture full 30-step trajectories for multiple prompts and seeds, fit schedule-specific momentum and neighbour-transport parameters, choose four anchors on training folds, and then run a genuinely branched sampler on held-out prompts. Capture must include the initial pre-denoiser noise latent, which the current callback-only 16-step store omits, so the first warp can be calibrated rather than falling back to raw Taylor.

Three references are required: native four-step generation, the full 30-step teacher, and four-call teleportation from the same initial noise. A candidate passes only if (i) denoiser calls and both denoiser-only and end-to-end latency are reported; (ii) every decoded panel is screened for collapse; (iii) PSNR, SSIM, and a perceptual metric quantify teacher retention; and (iv) blinded preference and prompt-adherence evaluation cannot distinguish it materially from the 30-step quality mode. A high PSNR alone is not the goal: a useful system may produce a different, equally strong composition.

The training objective should likewise combine endpoint consistency with decoded perceptual supervision:

$$\mathcal{L} = \lambda_z \|\tilde{x}_{a_{j+1}} - x_{a_{j+1}}\|_2^2 + \lambda_p \mathcal{L}_{\text{perc}}(D(\tilde{x}), D(x)) + \lambda_c \mathcal{L}_{\text{consistency}}, \tag{8}$$

where D is the VAE decoder. This connects the walker to consistency distillation without requiring the production denoiser itself to be replaced.

7.2 Four varied images with selective refinement

A common serving request generates four candidates for one prompt. Text encoding, model residency, and launch overhead are already shared, while four independent noise seeds provide baseline diversity. Latent teleportation adds a controlled axis: assign each lane a different high-ranking anchor schedule and, when available, a different compatible cached-neighbour prior. The four near-optimal schedules in Table 3 cost only 2.4% more latent error than the best single schedule in the offline proxy, so staggering need not mean choosing obviously weak paths.

At each anchor, the server compacts only lanes needing the big denoiser into a microbatch. Teleported lanes continue through scalar momentum, a resident small walker, or prefetched neighbour deltas. A later shared anchor can reconverge the lanes for an efficient full batch. This creates a natural quality–variation–compute control: conservative lanes receive more real refinement, exploratory lanes receive a different warp, and all four reuse the same prompt state. The production benchmark must report batch latency, active-lane sizes, pairwise perceptual diversity, prompt adherence, and per-lane quality; the current paper has not yet measured that GPU path.

7.3 How the accelerators combine

The methods operate at different layers, so several combinations are natural. They are not automatically multiplicative: once one method removes most denoiser work, Amdahl's law reduces the headroom of the next.

8 Serving Integration

The intended deployment target is a single-GPU multi-tenant server in which a diffusion worker shares a device with a language model, a background-removal worker and a 3D worker. Two pieces of that stack are relevant here and are open source [Penkman, 2026b].

First, admission and device-memory arbitration. Four processes sizing their own workloads from `cudaMemGetInfo` is a race: the read counts a caching allocator's retained blocks as used, and is stale the instant two tenants read it concurrently, so both size a batch against the same free bytes and the second runs out of memory. The gateway replaces this with a lease broker

Technique	What it contributes	Combination with latent teleportation
AYS	Places a small number of solver evaluations where their local approximation matters most.	Supplies the real anchors; teleportation predicts the latent state between them.
Flow matching / reflow	Trains straighter probability-flow trajectories and improves low-NFE integration.	Extends the horizon over which a tangent or neighbour transport remains valid; coefficients must still be refit.
Consistency / LCM	Learns a direct few-step map to a consistent endpoint.	Acts as a cheap proposer, endpoint teacher, or fallback lane; a native 2–4-step model leaves less post-training skip headroom.
DeepCache / FORA / TaylorSeer	Reuses or forecasts internal denoiser features across nearby steps.	Can reduce the cost of the remaining real anchors while latent teleportation reduces their count.
Fused kernels / compiled serving	Reduces each real call, allocation, transfer, and launch overhead.	Speeds the unavoidable anchors and makes active-lane compaction practical; measured total gain remains Amdahl-limited.
Trajectory retrieval	Provides prompt- and style-conditioned local directions from previous generations.	Adjusts the warp direction, curvature, and confidence rather than copying a finished image.

Table 8: Composability map. Rows describe mechanisms and hypotheses, not a benchmark claiming that all combinations have been measured.

— a tenant asks for headroom and is told a number it may rely on, with granted-but-not-yet allocated memory withheld from the next caller. This is what would allow a draft model to be resident at all, which is the natural next step for the speculative family.

The calibrated forecaster itself needs only a small table of $(t, k, \alpha_{t,k})$ scalars and should remain device-resident. Larger trajectory records belong in pinned host memory or NVMe, with prompt retrieval and asynchronous prefetch occurring while the text encoder or preceding anchor runs. The request scheduler should batch by model/LoRA identity first, then by next anchor, compact active lanes into contiguous buffers, and release cached trajectory pages as soon as their last consumer advances. Model or adapter swaps become natural preemption boundaries: finish an anchor, spill the small latent state, serve the higher-priority resident model, then restore and continue from a verified point.

Second, the request path itself is C rather than Python, with the HTTP relay preserving byte-exact streaming so that image bytes are never re-encoded. CuteDSL contributes fused kernels used by the Z-Image transformer on sm_120 — fused QKV projection with RoPE, fused QK-norm, RMS norm, SiLU-gated FFN and AdaLN — which accelerate the denoiser itself and are therefore complementary to everything in this paper: they make each step cheaper, whereas we tried to make steps fewer. Given the Amdahl result of Section 5.5, kernel work of this kind is currently the better investment of the two at our operating point. At a 30-step quality mode, however, denoising should occupy a much larger share of latency; step reduction, per-step fusion, and memory-aware scheduling can then compound over different portions of the request.

9 Limitations and Threats to Validity

The base model is already distilled. Z-Image Turbo is a few-step model. Step-reduction methods have far less to remove from an already-compressed schedule than from a 50-step sampler, and results here should not be extrapolated to undistilled checkpoints, where we would expect drafting to look considerably better.

Small prompt and seed set. Six prompts per configuration, one seed, and five prompts after warm-up exclusion for timing. The contact sheets expose every image rather than hiding this sample size, but they do not turn six examples into a population. This is enough to establish that the timing effect is near 1.0× and nowhere near enough to rank methods separated by a few percent. The method-by-style reversals at $k = 3$ are evidence against a universal ranking, not evidence for six distinct style distributions.

PSNR and SSIM are retention metrics, not quality judges. A drafted trajectory that lands on a different but equally plausible image is penalised as heavily as one that lands on mush. The qualitative artifacts show precisely this: many drafted images remain recognisable while differing compositionally from the baseline, whereas the skip arm's 8.9 dB correctly identifies noise. SSIM adds structural sensitivity but inherits the same reference dependence. A perceptual or preference-based metric is needed, and a visual-ranking model — for instance the Gemma model already resident in the serving stack, used as a pairwise judge — is one possible instrument. We have not run this, and we decline to report an aesthetic-quality ranking we have not measured.

The trajectory cache is not evaluated end-to-end here. The predictability and drafting results are measured; the k-NN teleportation results are not included in this report, and the production trajectory corpus (3D assets and an art library, on the order of tens of gigabytes) is described from the system design rather than measured in these experiments.

The aligned schedule is an offline true-anchor proxy. Table 3 resets every interval to a latent from the recorded teacher trajectory. A real branched sampler feeds an approximate teleported state into its next denoiser call, so errors can compound or be corrected in ways this table cannot represent. The 30-step/four-call target, active-lane batch compaction, pairwise diversity, and decoded images from calibrated momentum remain queued production experiments. We do not convert the proxy improvement into an image-quality or latency claim. Because the stored 16-step callbacks begin after the first denoiser call, the current deployment table also lacks a coefficient for the first interval; the implementation explicitly falls back to unit-scale Taylor there.

No exact verification. Unlike speculative decoding, our accepted latents are not guaranteed to match the target model's. Every method here trades quality for time, and the confidence gate bounds that trade heuristically rather than provably.

10 Conclusion

Latent teleportation is a family of approximate trajectory forecasts and warm starts for diffusion, not an exact verifier. The premise is supported: the trajectory is smooth enough that one schedule-calibrated momentum scalar reduces held-out forecast error by 20–37% over raw Taylor through $k = 8$. Aligning a four-interval proxy reduces local error by 35.3%, and four distinct near-optimal schedules stay within 2.4% of the best. These results support schedule-aware and batch-selective teleportation, while the evaluated drafting schemes remove up to 3.2× denoiser

calls. All 36 learned and Taylor candidates in the image sweep preserve the requested subject or scene, while larger horizons introduce visible retention costs that the grids make explicit.

At the tested 512^2/16-step serving point, fixed text and VAE work masks most of the saved calls: speculative walking measures 1.04× end to end and the destructive skip ceiling is 1.10×. That result identifies where the system must move next. The immediate milestone is a held-out 30-step teacher experiment with approximately four real calls, decoded-image and preference evaluation, and denoiser-only timing alongside end-to-end timing. A batch-of-four run should then test staggered schedules with active-lane compaction and pairwise diversity. Cache-guided teleportation, confidence gating, and LoRA-space transport then remain distinct follow-on hypotheses rather than implied results.

The corrected warm-up analysis is equally important: the apparent 1.5× became 1.04× when a cold first iteration was removed. Publishing both the promising forecasting signal and the actual serving constraint gives future tuning a reliable baseline.

Reproducibility

Code, the analysis harnesses, generated contact sheets, their source hashes, and the raw result JSON are in Penkman [2026a]. Full-resolution source panels remain in the experiment store; the committed grids contain every panel used in the paper. The serving gateway is in Penkman [2026b]. The LoRA adapter library is not open source; its integration mechanism is described in Section 3.5.

References

Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwei Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint*, 2023.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. In *IEEE Transactions on Big Data*, 2019.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint*, 2024.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

Jiacheng Liu, Chang Zou, Yuanhuiyi Lyu, Jiahui Chen, and Linfeng Zhang. From reusing to forecasting: Accelerating diffusion models with TaylorSeers. *arXiv preprint*, 2025. Referred to in this paper as CG-Taylor after the confidence-gated variant used in the CuteDSL codebase.

Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations (ICLR)*, 2023.

Xingchao Liu, Xiwen Zhang, Jianzhu Ma, Jian Peng, and Qiang Liu. InstaFlow: One step is enough for high-quality diffusion-based text-to-image generation. In *International Conference on Learning Representations (ICLR)*, 2024.

Simian Luo, Yiqin Tan, Longbo Huang, Jian Li, and Hang Zhao. Latent consistency models: Synthesizing high-resolution images with few-step inference. *arXiv preprint*, 2023.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. DeepCache: Accelerating diffusion models for free. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

Lee Penkman. CuteDSL: Accelerated model inference via custom CuTeDSL and CUDA kernels. https://github.com/lee101/cutedsl, 2026a.

Lee Penkman. omniserve-native: A native c inference gateway for a single GPU. Apache-2.0 licensed source release, 2026b.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Amirmojtaba Sabour, Sanja Fidler, and Karsten Kreis. Align your steps: Optimizing sampling schedules in diffusion models. In *International Conference on Machine Learning (ICML)*, 2024.

Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. In *International Conference on Learning Representations (ICLR)*, 2022.

Axel Sauer, Dominik Lorenz, Andreas Blattmann, and Robin Rombach. Adversarial diffusion distillation. *arXiv preprint*, 2023.

Apoorv Saxena. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023.

Pratheba Selvaraju, Tianyu Ding, Tianyi Chen, Ilya Zharkov, and Luming Liang. FORA: Fast-forward caching in diffusion transformer acceleration. *arXiv preprint*, 2024.

Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. In *International Conference on Machine Learning (ICML)*, 2023.

Tongyi Lab, Alibaba Group. Z-image: An efficient foundation model for image generation. Model card and release notes, 2025.